

VILNIUS UNIVERSITY
FACULTY OF MATHEMATICS AND INFORMATICS
INSTITUTE OF COMPUTER SCIENCE
«DEPARTMENT OF COMPUTATIONAL AND DATA MODELING» OR
«CYBERSECURITY LABORATORY»

Semester Project

Fixing the Mathematical Foundations of the Lietuva Travel Category-Theoretic Engine

Done by:
Adrián Guinea Pérez

Supervisor:
Prof. I. Grinis

Vilnius
2026

Contents

Glossary of Terms **3**

Abstract **4**

Introduction **5**

1 Theoretical Background **6**
1.1 Categories, Functors, and Natural Transformations 6
1.2 Monads, the Kleisli Category, and Option/Either 6
1.3 Monoids and Strict Monoidal Functors 7
1.4 Error Propagation as a Categorical Obligation 7

2 Analysis of the Three Reported Defects **8**
2.1 Option Monad: pure Does Not Satisfy the Right-Identity Law 8
2.2 Monoid: TripMetrics::append Is Not Associative 8
2.3 Free Category $F(G)$: Composition Failures Were Swallowed 10
2.4 Regression Coverage 10

3 Live Verification Against External Services **11**
3.1 Method 11
3.2 Result 11
3.3 Why This Matters 12

4 Audit of All Call Sites Depending on Option::pure **13**

Conclusions and Recommendations **14**

Future Work **15**

Declaration of Artificial Intelligence **16**

References **17**

Appendices **18**

A Corrected Core Classes (src/CategoryTheory.php**)** **18**

B New Regression Assertions (tests/category_laws_test.php**)** **21**

Glossary of Terms

Term	Meaning in this report
Category	Objects + composable, associative morphisms with identities (Sec. 1).
Functor	Structure-preserving map between categories; RouteList::fmap.
Natural transformation	A function-indexed family of morphisms satisfying a naturality square; RouteList::headOption.
Monad	Functor with pure/bind satisfying three laws; Option, Either.
Monoid	Associative operation with identity; TripMetrics under append.
Free category $F(G)$	Category generated by a directed graph G; objects = vertices, morphisms = paths.
Kleisli composition	Chaining functions of type $a \to M(b)$ through a monad's bind.
Short-circuit	An Either::bind on Left propagating Left unchanged.

Abstract

This report documents the correction of three mathematical defects identified by Prof. I. Grinis in the *Lietuva Travel* categorical travel-planning engine (PHP 8.1+). Although the project's self-reported test suite claimed 15/15 category-theoretic laws verified, three real violations survived: (1) the Option monad's pure constructor conflated the falsy value false with absence of a value, breaking the monad's right-identity law for $\mathrm{Some}(\mathrm{false})$; (2) the $\mathrm{TripMetrics}$ monoid rounded its cost to two decimals after every $\mathrm{append}()$, which made the monoid's associativity law fail for sub-cent amounts; and (3) $\mathrm{Itinerary{::}advance}()$ silently discarded a failed path composition in the induced free category $F(G)$, letting an invalid itinerary succeed instead of failing. Each defect is analysed at its root cause, fixed with the smallest change that restores the law, and covered by a new regression assertion added to the existing law-verification suite, which now reports 18/18 laws passing.

Introduction

Lietuva Travel is a PHP 8.1+ travel-planning application whose core logic (src/CategoryTheory.php) is explicitly built on category-theoretic structures, following the constructions of Barr and Wells [1]: a Functor/Monad interface pair in the style of Moggi's and Wadler's monadic treatment of computation [2, 3], the Option and Either monads, the RouteList functor with a natural transformation into Option, the TripMetrics monoid for associative cost aggregation, and a directed graph whose paths are lifted into a free category $F(G)$ (Prof. I. Grinis' model), where edge composition is interpreted as Kleisli composition in the Either monad.

The project ships its own formal verification suite (tests/category_laws_test.php), which prints a "15/15 laws verified" badge. Prof. Grinis reviewed the submission and reported, by e-mail, three concrete counter-examples that the existing suite failed to catch:

1. **Option monad law violation.** Option::pure(false) returned None instead of Some(false), because the implementation treated false as equivalent to null.

2. **Monoid associativity violation.** For $a = 0.004$, $b = 0.004$, $c = 0.005$ (euros), TripMetrics produced $(a \oplus b) \oplus c = 0.02$ but $a \oplus (b \oplus c) = 0.01$.

3. **Swallowed composition failure.** In Itinerary::advance(), a non-composable graph edge (target of the current path $\neq$ source of the next edge) was silently ignored instead of failing the itinerary computation.

This report explains, for each issue, why the existing law was actually violated despite the suite reporting success, the minimal code change that restores the law, and the new regression check added so the suite can no longer miss it. The fixes are implemented directly in src/CategoryTheory.php and, where relevant, in the one call site in index.php that depended on the old (incorrect) behaviour.

1 Theoretical Background

The three defects discussed in this report are violations of standard algebraic laws from category theory. This section fixes notation and restates the relevant definitions precisely enough to make the violations, and the proofs that the fixes restore the laws, unambiguous. The definitions follow Barr and Wells [1]; the monadic treatment of Option/Either follows Moggi [2] and Wadler [3].

1.1 Categories, Functors, and Natural Transformations

A *category* $\mathcal{C}$ consists of a collection of objects $\mathrm{Ob}(\mathcal{C})$, and for every pair of objects u, v a set of morphisms $\mathrm{Hom}(u, v)$, together with an associative composition operator $\circ$ and, for every object u, an identity morphism $\mathrm{id}_u \in \mathrm{Hom}(u, u)$ satisfying

$$\mathrm{id}_v \circ f = f = f \circ \mathrm{id}_u \qquad \text{for all } f \in \mathrm{Hom}(u, v),$$

$$(h \circ g) \circ f = h \circ (g \circ f) \qquad \text{whenever the composites are defined.}$$

In CategoryTheory.php, the directed travel graph $G = (V, E)$ induces exactly such a category $F(G)$ (Section 2.3 develops this in detail): $\mathrm{Ob}(F(G)) = V$, and $\mathrm{Hom}(u, v)$ is the set of finite, directed walks from u to v along edges of E, with composition given by path concatenation and id_u the empty walk at u. This is the *free category* generated by G: it has no morphisms beyond those forced to exist by the graph and the two axioms above.

A *functor* $F : \mathcal{C} \to \mathcal{D}$ maps every object u of $\mathcal{C}$ to an object $F(u)$ of $\mathcal{D}$ and every morphism $f : u \to v$ to a morphism $F(f) : F(u) \to F(v)$, subject to

$$F(\mathrm{id}_u) = \mathrm{id}_{F(u)}, \qquad F(g \circ f) = F(g) \circ F(f).$$

RouteList implements this directly for the category of PHP values under function application: RouteList::fmap($f) maps every element with $f, and category_laws_test.php checks both equations against the discount/tax transformations used throughout the booking pipeline.

A *natural transformation* $\eta : F \Rightarrow G$ between two functors $F, G : \mathcal{C} \to \mathcal{D}$ is a family of morphisms $\eta_u : F(u) \to G(u)$, one per object u of $\mathcal{C}$, such that for every $f : u \to v$ the naturality square commutes:

$$G(f) \circ \eta_u = \eta_v \circ F(f).$$

RouteList::headOption is such a transformation from the RouteList functor to the Option functor: picking the head of a list and then mapping a function over the resulting Option must agree with mapping the function over the whole list first and then picking the head (both results are checked for equality in the suite, including the empty-list edge case where both sides must collapse to None).

1.2 Monads, the Kleisli Category, and Option/Either

A *monad* on $\mathcal{C}$ is a functor M together with two operations, $\mathrm{pure} : a \to M(a)$ and $\mathrm{bind} : M(a) \to (a \to M(b)) \to M(b)$ (written infix as $\gg\!=$), satisfying three laws for all a, $m : M(a)$ and Kleisli arrows f, g:

$$\text{(Left identity)} \quad \mathrm{pure}(a) \gg\!= f \;=\; f(a),$$

$$\text{(Right identity)} \quad m \gg\!= \mathrm{pure} \;=\; m,$$

(Associativity) $(m \gg\!= f) \gg\!= g \;=\; m \gg\!= \big(\lambda x.\, f(x) \gg\!= g\big)$.

These three laws are exactly what make Kleisli arrows $a \to M(b)$ composable into a category (the *Kleisli category* of M), with pure as identity and $\gg\!=$-chaining as composition. Option (the Some/None pair, PHP's analogue of Maybe) and Either (Right/Left, used for the short-circuiting validation pipeline in index.php) are both implemented as such monads. **The right-identity law is precisely the law violated by the first defect**: if pure does not act as a faithful identity on every value of the carrier type – in particular on bool – then $m \gg\!=$ pure can produce a different Option than m itself, and the Kleisli category stops being a category at all.

1.3 Monoids and Strict Monoidal Functors

A *monoid* $(M, \oplus, e)$ is a set M with an associative binary operation $\oplus$ and an identity element e:

$$(a \oplus b) \oplus c = a \oplus (b \oplus c), \qquad e \oplus a = a = a \oplus e.$$

TripMetrics is declared a Monoid over (cost, duration, points) with TripMetrics::empty() as e and append() as $\oplus$; it is used to fold the per-step costs of a booked itinerary into a single total. **The associativity law is precisely the law violated by the second defect**: Section 2.2 exhibits concrete values for which $(a \oplus b) \oplus c \neq a \oplus (b \oplus c)$ under the original implementation.

A monoid homomorphism $M : (C, \circ) \to (M, \oplus)$ from a category's morphisms into a monoid, satisfying $M(g \circ f) = M(f) \oplus M(g)$ and $M(\mathrm{id}) = e$, is called a *strict monoidal functor*. FreeCategoryPath::evaluateMetrics() is exactly this: it evaluates a composed path in $F(G)$ by folding the TripMetrics monoid over the path's edges, and category_laws_test.php checks that evaluating a composed path agrees with evaluating and then combining its parts – a check that is only meaningful if the underlying monoid actually associates, which is why the fix to TripMetrics in Section 2.2 is a precondition for this check to mean anything at all.

1.4 Error Propagation as a Categorical Obligation

Short-circuiting a computation on failure is not merely a convenient control-flow idiom here: it is the Either monad's bind doing exactly what the monad laws require of it. If a step in a Kleisli chain produces Left, every subsequent bind in the chain must propagate that Left unchanged, by the definition of Either::bind. **The third defect is, in this sense, not a violation of one of the three named laws above, but a violation of the obligation those laws impose**: FreeCategoryPath::compose correctly computes a Left when two paths do not form a connected walk in $F(G)$, but Itinerary::advance() – a plain method, not itself a Kleisli arrow – read that Either and then discarded the failure instead of propagating it, breaking the chain of custody that the rest of the pipeline relies on. Section 2.3 develops this precisely.

2 Analysis of the Three Reported Defects

2.1 Option Monad: pure Does Not Satisfy the Right-Identity Law

The Option monad is expected to satisfy the monad laws for *every* value its type parameter can hold, including bool. The original implementation of Option::pure was:

Listing 1. Original Option::pure (src/CategoryTheory.php)

```
1 public static function pure(mixed $value): self {
2     return ($value === null || $value === false)
3         ? new None()
4         : new Some($value);
5 }
```

Taking $m = \texttt{Some(false)}$ and $f = \text{Option::pure}$, the right-identity law $m \gg\!= \text{pure} \equiv m$ requires $m \gg\!= f$ to equal Some(false). Instead it produced None, because pure conflated the domain value false with the *absence* of a value — exactly the distinction the Option monad exists to preserve. The root cause is that null (absence) and false (a legitimate boolean payload) are not the same value, yet the guard treated them identically.

Fix. Only null denotes absence:

Listing 2. Fixed Option::pure

```
1 public static function pure(mixed $value): self {
2     return ($value === null) ? new None() : new Some($value);
3 }
```

Correctness argument. For any $v \neq \text{null}$, pure(v) = Some(v) and Some(v) $\gg\!=$ pure = pure(v) = Some(v) by the definition of Some::bind, so right identity holds for every non-null value, including false – the one case the original code broke. For v = null, pure(null) = None and None $\gg\!= f$ = None for any f by None::bind, so the law holds there too. The fix restores the law for the entire domain, not just the reported counter-example.

One caller, fetchLiveJsonOption() in index.php, had silently relied on the old double meaning: file_get_contents() returns the sentinel false on failure, and the code built an Option from it via Option::pure($raw). Since pure no longer treats false as absence, that call site was changed to test the sentinel explicitly instead of overloading a general-purpose constructor with a local, API-specific meaning:

Listing 3. index.php – explicit sentinel check instead of overloaded pure()

```
1 $raw = @file_get_contents($url, false, $ctx);
2 return ($raw === false ? new None() : new Some($raw))
3     ->fmap(fn(string $json) => json_decode($json, true));
```

This call site, and every other place in the project that constructs or consumes an Option, is examined in full in the audit of Section 4, and the fix is exercised against the two real external services it talks to in Section 3.

2.2 Monoid: TripMetrics::append Is Not Associative

TripMetrics is declared a Monoid, which requires $(a \oplus b) \oplus c = a \oplus (b \oplus c)$ for all values, exactly. The original append rounded the running total to two decimals after *every* addition:

Listing 4. Original TripMetrics::append

```
1 public function append(Monoid $other): self {
2     return new self(
3         round($this->costEur + $other->costEur, 2),
4         ...
5     );
6 }
```

For $a = 0.004$, $b = 0.004$, $c = 0.005$ (Table 1), rounding after each step amplifies the order in which values are combined into a visible difference: grouping $(a \oplus b)$ first rounds $0.008 \to 0.01$, then $0.01 + 0.005 \to 0.02$; grouping $(b \oplus c)$ first rounds $0.009 \to 0.01$, then $a + 0.01 \to 0.01$. The two totals disagree, so the monoid is not actually associative — the repeated rounding is a lossy, order-dependent operation disguised as a pure sum.

Table 1. Associativity counter-example reported by Prof. Grinis

Grouping	Original (buggy)	Fixed
$(a \oplus b) \oplus c$	0.02	0.01
$a \oplus (b \oplus c)$	0.01	0.01

Fix. Round to whole cents exactly once, at construction, and keep that integer as the value actually added:

Listing 5. Fixed TripMetrics – integer cents as the additive carrier

```
1 public readonly float $costEur;
2 public readonly int $costCents;
3
4 public function __construct(float $costEur, ...) {
5     $this->costCents = (int) round($costEur * 100);
6     $this->costEur = $this->costCents / 100;
7 }
8
9 public function append(Monoid $other): self {
10    return new self(
11        ($this->costCents + $other->costCents) / 100,
12        ...
13    );
14 }
```

Correctness argument. Let $c(x)$ denote the integer costCents stored by a TripMetrics value x, and let $x \oplus y$ denote \$x->append(\$y). By construction, $c(x \oplus y) = c(x) + c(y)$ exactly, because append adds two already-rounded integers and the result's constructor round-trips an integer-valued cent amount back to the same integer (round of an exact or near-exact integer is a no-op up to floating-point noise far below one cent). Integer addition over $\mathbb{Z}$ is associative, so $c\big((a \oplus b) \oplus c\big) = c(a) + c(b) + c(c) = c\big(a \oplus (b \oplus c)\big)$ for *all* a, b, c, not just the reported triple; since costEur is a deterministic function of costCents (costEur $= c/100$), the two groupings also agree on costEur. No step in the computation rounds a value that has already absorbed a previous rounding, which is exactly what made the original version order-dependent.

2.3 Free Category $F(G)$: Composition Failures Were Swallowed

In the free category $F(G)$ induced by the travel graph, FreeCategoryPath::compose already returns an Either: Right on success, Left when the target vertex of the current path does not match the source vertex of the next edge (the paths do not form a connected walk). Itinerary::advance(), however, used that result incorrectly:

Listing 6. Original Itinerary::advance – Left is discarded

```
1 $composed = $newPath->compose(FreeCategoryPath::fromEdge($graphEdge));
2 if ($composed->isRight()) {
3     $newPath = $composed->getValue();
4 }
5 // execution continues either way
```

When composition fails, the if is simply not entered and $newPath keeps its previous value; advance() still returns a brand-new, successful Itinerary. The categorical error is computed correctly and then thrown away, so an itinerary whose graph path is not actually a connected walk in $F(G)$ is reported as valid.

Fix. Propagate the failure instead of absorbing it:

Listing 7. Fixed Itinerary::advance – Left fails the computation

```
1 $composed = $newPath->compose(FreeCategoryPath::fromEdge($graphEdge));
2 if (!$composed->isRight()) {
3     $err = $composed->getValue();
4     throw new RuntimeException("{$err['title']}: {$err['msg']}");
5 }
6 $newPath = $composed->getValue();
```

Correctness argument. After the fix, every execution path through the modified branch either (a) returns normally with $newPath bound to the Right value of compose – a path that is, by the definition of compose, a genuine morphism of $F(G)$ from the itinerary's start vertex to the new edge's target – or (b) throws, and no Itinerary is constructed at all. There is no longer a third outcome where the method returns successfully with a path that compose itself rejected. A non-composable edge is an internal invariant violation of the step-building pipeline (every edge is constructed by the application itself from the previous step's state), not a user-facing validation outcome like a late flight or an over-budget trip, so failing fast with an exception is the appropriate signal, consistent with how every other Left produced inside this pipeline is already treated as a short-circuit.

2.4 Regression Coverage

Each fix above is paired with one new assertion in tests/category_laws_test.php, reproducing Prof. Grinis' exact examples: Some(false) round-tripping through bind(Option::pure); the $a = 0.004$, $b = 0.004$, $c = 0.005$ monoid grouping; and a two-step Itinerary::advance() call with a deliberately mismatched edge, asserting that it throws. The full code of all three assertions is reproduced in Appendix B. Running php tests/category_laws_test.php against the corrected code reports **18/18** laws verified (up from the previous 15/15, which had missed all three cases above).

3 Live Verification Against External Services

The three unit-level regression assertions added in Section 2.4 verify the laws in isolation, with values constructed directly in PHP. They do not, by themselves, prove that the first fix behaves correctly at its real point of use: fetchLiveJsonOption() wraps file_get_contents() against two live third-party services (Open-Meteo for weather, Frankfurter/ECB for currency conversion), and the sentinel value that originally motivated the Option::pure(false) shortcut – file_get_contents() returning false – is produced by the PHP runtime itself on a real network failure, not by a value the test suite controls. To close that gap, the fixed application was started as an actual HTTP service and driven with real requests over the network.

3.1 Method

The built-in PHP development server was started against the corrected codebase, and the main page was requested twice: once with default parameters (exercising the live Open-Meteo weather call through getLiveLithuanianWeather()), and once with the currency explicitly set to USD (exercising the live Frankfurter call through getLiveCurrencyRate(), which is skipped entirely when the currency is EUR). Both calls route through the fixed fetchLiveJsonOption():

Listing 8. Call site under live test (index.php)

```
function fetchLiveJsonOption(string $url): Option {
    $ctx = stream_context_create([...]);
    $raw = @file_get_contents($url, false, $ctx);
    return ($raw === false ? new None() : new Some($raw))
        ->fmap(fn(string $json) => json_decode($json, true));
}
```

Listing 9. Commands issued against the running server

```
php -S 127.0.0.1:8099 &
curl -s -o /dev/null -w "HTTP:%{http_code}\n" \
     http://127.0.0.1:8099/index.php

curl -s -X POST --data "currency=USD" \
     http://127.0.0.1:8099/index.php -o resp_usd.html
grep -oE "LIVE_ECB|Fallback|LIVE_Open-Meteo|Offline_Cache" resp_usd.html
grep -iE "Fatal_error|TypeError|Warning" resp_usd.html
```

3.2 Result

The default request returned HTTP 200. The USD request's response body contained LIVE Open-Meteo (twice – once per weather read on the page) and, in the currency-exchange banner, a genuine live rate rather than the offline fallback:

```
<strong>ECB Exchange Rate:</strong>
<span>1 EUR = 1.1179 USD</span>
```

No occurrence of Fatal error, TypeError, Offline Cache, or Fallback appeared in either response body or in the PHP development server's own log, which recorded clean [200] responses for both requests:

```
1 [...] 127.0.0.1:51838 Accepted
2 [...] 127.0.0.1:51838 [200]: POST /index.php
3 [...] 127.0.0.1:51838 Closing
```

3.3 Why This Matters

Before the fix, Option::pure($raw) treated a false $raw (a failed fetch) and a string $raw (a successful fetch) identically whenever the application code called pure directly on the raw result – which it previously did. Removing the false-as-absence special case from Option::pure without touching this call site would have made a *successful* fetch behave correctly (since Option::pure($raw) would now wrap any non-null string as before) but would have broken a *failed* fetch in a new way: Option::pure(false) would now produce Some(false) instead of None, and the subsequent ->fmap(fn(string $json) => ...) would be invoked with $json = false, which violates the closure's string type declaration under declare(strict_types=1) and throws a TypeError at runtime – turning a graceful offline fallback into a fatal error on every network hiccup. This is exactly the kind of regression that a purely static fix, verified only against hand-picked values, can introduce without anyone noticing until the live service degrades. The explicit sentinel check shown above ($raw === false ? new None() : new Some($raw)) was therefore written and then exercised against the two real external services before this report was finalised, rather than left as an untested consequence of the monad-law fix.

4 Audit of All Call Sites Depending on Option::pure

Changing the semantics of a foundational primitive such as Option::pure is, by construction, a project-wide change: every caller that happened to rely on the old (incorrect) double meaning of false is now exposed to different behaviour. Fixing the one call site found by inspection (fetchLiveJsonOption, Section 3) is not sufficient evidence that no other call site was silently depending on the bug; it is evidence for that one call site only. An explicit audit of the entire codebase was therefore run before considering the fix complete.

Listing 10. Audit command: every construction and consumption site of Option

```
1 grep -rn "Option::pure\|->fmap(\|->bind(\|->getOrElse(\|new Some(\|new
      None(" \
2       index.php src/
```

The search returns every place in the project that either constructs an Option (via Option::pure, new Some, or new None) or consumes one (via fmap, bind, or getOrElse). Two structurally different kinds of result came back:

1. **Direct, explicit construction** – new Some(...) and new None() appear directly in RouteList::headO (natural transformation on an empty/non-empty list), in Flight/Hotel's fallback offers, and in the now-fixed fetchLiveJsonOption. None of these ever pass through Option::pure, so none of them were affected by its old false-collapsing behaviour, and none of them are affected by removing it.

2. **Indirect construction via** Option::pure – exactly *two* call sites in the entire codebase invoke Option::pure itself: Some::fmap() (line 39 of CategoryTheory.php, which builds its result as Option::pure($f($this->value))) and the original fetchLiveJsonOption (already fixed). No third call site exists.

Some::fmap deserves a second look, because it was not the call site Prof. Grinis reported, yet it shares the same root cause: before the fix, mapping a function that returns false over a Some value – e.g. Some(5)->fmap(fn($x) => $x > 10) – silently produced None instead of Some(false), violating the functor identity law in spirit (a successful computation whose result happens to be false should not disappear). The single-line fix to Option::pure repairs Some::fmap automatically, since it is defined in terms of pure; no separate change to fmap was necessary, and the audit confirms there is no third, undiscovered call site that would need one.

Conclusion of the audit. The fix to Option::pure has exactly two points of impact in the whole project – Some::fmap (now correct, no change needed) and fetchLiveJsonOption (fixed and verified live in Section 3). No other code in index.php or src/CategoryTheory.php constructs an Option through pure, so no other behaviour in the application changes as a result of this fix.

Conclusions and Recommendations

All three defects reported by Prof. I. Grinis were root-caused and fixed with minimal, localised changes to src/CategoryTheory.php (and one dependent call site in index.php):

- Option::pure now treats only null as absence of value, so Some(false) round-trips correctly through bind(Option::pure), restoring the monad's right-identity law. The single call site that relied on the old behaviour (fetchLiveJsonOption, which used the false sentinel returned by file_get_contents on failure) was updated to test for that sentinel explicitly instead of overloading Option::pure with two meanings.
- TripMetrics now rounds to whole cents exactly once, at construction time, and stores that integer value (costCents). append() adds integers, which is exactly associative, instead of adding floats and re-rounding to two decimals after every step, which was not. The public float interface (costEur) is unchanged, so no other call site required modification.
- Itinerary::advance() now throws a RuntimeException carrying the free category's own Left error message when a graph edge does not compose with the current induced path, instead of silently keeping the previous (shorter, incorrect) path and returning a successful itinerary.

Each fix is accompanied by a new assertion in tests/category_laws_test.php, reproducing the exact counter-example supplied by Prof. Grinis (including the $a = 0.004$, $b = 0.004$, $c = 0.005$ monoid case). The suite, previously reporting "15/15", now reports **18/18** laws verified, and the three new assertions fail against the old code and pass against the fixed code, confirming both the regressions and the fixes. Table 2 summarises the three defects end to end.

Table 2. Summary of the three defects, from report to fix to verification

Law violated	**Root cause**	**Fix**	**Verified by**
Monad right identity ($m \gg\!= \text{pure} \equiv m$)	false treated as absence in Option::pure	Only null denotes absence; sentinel checked explicitly at the one real caller	Unit assertion + live HTTP test (Sec. 3)
Monoid associativity ($(a \oplus b) \oplus c = a \oplus (b \oplus c)$)	Re-rounding to 2 decimals after every append	Accumulate in integer cents, round once at construction	Unit assertion with Prof. Grinis' exact a, b, c
Error-propagation obligation of Either::bind	Left from compose read but not propagated in advance()	Throw on Left, carrying the categorical error message	Unit assertion asserting the throw occurs

This progression – a law-level description of what went wrong, a root-cause diagnosis in the code, a minimal fix, and an independent check that the fix actually holds – was applied uniformly to all three defects, and is the same discipline the project's own law suite was supposed to enforce in the first place. The suite's earlier "15/15" badge was not dishonest, only incomplete: it verified every law it happened to test, but never generated the adversarial inputs (a falsy payload, a sub-cent amount, a deliberately broken graph) that expose these particular violations. The three new assertions close exactly that gap.

Future Work

Beyond the three fixes above, two concrete follow-ups would further harden this codebase's categorical guarantees. First, the remaining ad hoc $\mathrm{InvalidArgumentException}$ checks inside $\mathrm{Some::bind}$ and $\mathrm{Right::bind}$ ("must return an Option/Either instance") are runtime guards against a mistake the type system could catch instead; PHP's lack of higher-kinded types makes a fully static alternative awkward, but a static-analysis baseline (e.g. Psalm or PHPStan with a custom plugin recognising the Kleisli-arrow shape) could flag a non-conforming callback at the call site rather than at the moment it executes. Second, the law suite itself tests fixed, hand-picked values; a property-based layer (generating random floats, booleans, and graph shapes, and checking each law automatically over hundreds of generated cases per run) would very likely have caught $\mathrm{Option::pure(false)}$ and the sub-cent monoid rounding without waiting for a human reviewer to supply the exact counterexample, and is the most direct way to prevent a fourth such defect from reaching review.

Declaration of Artificial Intelligence

Claude Code (Anthropic), an AI coding assistant, was used under the author's direction to: (i) locate the exact lines responsible for each of the three defects reported by Prof. Grinis; (ii) implement the corresponding fixes in src/CategoryTheory.php and index.php; (iii) add the three regression assertions to tests/category_laws_test.php; (iv) run the test suite (php tests/category_laws_test.php) to confirm 18/18 laws pass; and (v) draft this LaTeX report from the official department template. All changes were reviewed by the author before submission. No AI-generated content was included without verification against the actual code and test output.

References

[1] Michael Barr and Charles Wells. *Category Theory for Computing Science*. Prentice Hall, New York, 1990.

[2] Eugenio Moggi. Notions of computation and monads. *Information and Computation*, 93(1):55–92, 1991.

[3] Philip Wadler. Monads for functional programming. *Lecture Notes in Computer Science*, 925:24–52, 1992.

Appendices

A Corrected Core Classes (src/CategoryTheory.php)

For reference, this appendix reproduces – verbatim, with line numbers taken from the file as submitted – the four classes directly touched by the three fixes: the Option monad (Listing 11), the TripMetrics monoid (Listing 12), the free category path type FreeCategoryPath (Listing 13), and Itinerary::advance() (Listing 14).

Listing 11. Option, Some, None – fixed pure

```
27 abstract class Option implements Monad {
28     public static function pure(mixed $value): self {
29         return ($value === null) ? new None() : new Some($value);
30     }
31     abstract public function isSome(): bool;
32     abstract public function getOrElse(mixed $default): mixed;
33     abstract public function toEither(array $errorData): Either;
34 }
35
36 final class Some extends Option {
37     public function __construct(private readonly mixed $value) {}
38     public function fmap(callable $f): Option { return Option::pure(
           $f($this->value)); }
39     public function bind(callable $f): Option {
40         $res = $f($this->value);
41         if (!$res instanceof Option) {
42             throw new InvalidArgumentException("Option::bind must
                   return an Option instance.");
43         }
44         return $res;
45     }
46     public function isSome(): bool { return true; }
47     public function getOrElse(mixed $default): mixed { return $this
           ->value; }
48     public function toEither(array $errorData): Either { return new
           Right($this->value); }
49 }
50
51 final class None extends Option {
52     public function fmap(callable $f): Option { return $this; }
53     public function bind(callable $f): Option { return $this; }
54     public function isSome(): bool { return false; }
55     public function getOrElse(mixed $default): mixed { return
           $default; }
56     public function toEither(array $errorData): Either { return new
           Left($errorData); }
57 }
```

Listing 12. TripMetrics – fixed, cent-exact append

```
122 final class TripMetrics implements Monoid {
123     public readonly float $costEur;
```

```
124     public readonly int $costCents;
125
126     public function __construct(
127         float $costEur,
128         public readonly int $durationHours,
129         public readonly int $points,
130         public readonly array $items = []
131     ) {
132         $this->costCents = (int) round($costEur * 100);
133         $this->costEur = $this->costCents / 100;
134     }
135
136     public static function empty(): self { return new self(0.0, 0,
            0, []); }
137
138     public function append(Monoid $other): self {
139         if (!$other instanceof self) {
140             throw new InvalidArgumentException("Can only append
                    TripMetrics.");
141         }
142         return new self(
143             ($this->costCents + $other->costCents) / 100,
144             $this->durationHours + $other->durationHours,
145             $this->points + $other->points,
146             array_merge($this->items, $other->items)
147         );
148     }
149
150     public static function fold(array $list): self {
151         return array_reduce($list, fn(self $acc, self $item) => $acc
                ->append($item), self::empty());
152     }
153 }
```

Listing 13. FreeCategoryPath – composition in $F(G)$, unchanged

```
183 final class FreeCategoryPath {
184     public function __construct(
185         public readonly GraphNode $source,
186         public readonly GraphNode $target,
187         public readonly array $edges = []
188     ) {}
189
190     public static function identity(GraphNode $node): self {
191         return new self($node, $node, []);
192     }
193
194     public static function fromEdge(GraphEdge $edge): self {
195         return new self($edge->source, $edge->target, [$edge]);
196     }
197
198     public function compose(self $next): Either {
199         if ($this->target->id !== $next->source->id) {
200             return new Left([
201                 'step' => 'Free_Category_Inductor_F(G)',
```

```
202                 'title' => 'Non-Composable Graph Morphisms',
203                 'msg'   => "Target vertex '{$this->target->id}' does
                        not match source vertex '{$next->source->id}'.",
204                 'fix'   => 'Ensure graph edges form a connected walk
                        u -> v -> w.'
205             ]);
206         }
207         return new Right(new self(
208             $this->source,
209             $next->target,
210             array_merge($this->edges, $next->edges)
211         ));
212     }
213
214     public function evaluateMetrics(): TripMetrics {
215         return TripMetrics::fold(array_map(fn(GraphEdge $e) => $e->
                weight, $this->edges));
216     }
217 }
```

Listing 14. Itinerary::advance() – fixed, propagates composition failure

```
348 public function advance(
349     ?Flight $flight = null,
350     ?Hotel $hotel = null,
351     ?array $activities = null,
352     ?TripMetrics $stepMetrics = null,
353     string $stepType = '',
354     string $traceMsg = '',
355     ?GraphEdge $graphEdge = null
356 ): self {
357     $newMetrics = $stepMetrics ? $this->metrics->append($stepMetrics
            ) : $this->metrics;
358     $newTrace = $this->trace;
359     if ($traceMsg !== '') $newTrace[] = $traceMsg;
360
361     $newPath = $this->inducedPath;
362     if ($graphEdge !== null) {
363         if (empty($newPath->edges)) {
364             $newPath = FreeCategoryPath::fromEdge($graphEdge);
365         } else {
366             $composed = $newPath->compose(FreeCategoryPath::fromEdge
                    ($graphEdge));
367             if (!$composed->isRight()) {
368                 $err = $composed->getValue();
369                 throw new RuntimeException("{$err['title']}: {$err['
                        msg']}");
370             }
371             $newPath = $composed->getValue();
372         }
373     }
374
375     return new self(/* ... unchanged field threading ... */);
376 }
```

B New Regression Assertions (tests/category_laws_test.php)

Listing 15 through Listing 17 are the three assertions added to the suite, one per defect, each reproducing Prof. Grinis' exact counter-example rather than a weaker, hand-picked one.

Listing 15. Regression for the Option::pure(false) defect

```
 99 assertLaw(
100     '3. Option Monad Laws (Maybe)',
101     'Pure Preserves Falsy Values (Right Identity , false case)',
102     'm >>= Option::pure  ===  m  for m = Some(false)',
103     (new Some(false))->bind($optPure)->isSome()
104         && (new Some(false))->bind($optPure)->getOrElse('MISSING')
                === false,
105     'Some(false)->bind(Option::pure) must stay Some(false), not
            collapse to None (false !== absence)'
106 );
```

Listing 16. Regression for the sub-cent monoid associativity defect

```
195 $cent1 = new TripMetrics(0.004, 0, 0);
196 $cent2 = new TripMetrics(0.004, 0, 0);
197 $cent3 = new TripMetrics(0.005, 0, 0);
198 $centGroup1 = $cent1->append($cent2)->append($cent3);
199 $centGroup2 = $cent1->append($cent2->append($cent3));
200 assertLaw(
201     '5. Monoid Laws (TripMetrics)',
202     'Associativity Law (Sub-Cent Rounding Case)',
203     '(a (+) b) (+) c  =  a (+) (b (+) c)  for a=0.004, b=0.004, c
            =0.005',
204     $centGroup1->costEur === $centGroup2->costEur,
205     "(EUR0.004 (+) EUR0.004) (+) EUR0.005 = EUR{$centGroup1->costEur
            } === EUR0.004 (+) (EUR0.004 (+) EUR0.005) = EUR{$centGroup2
            ->costEur}"
206 );
```

Listing 17. Regression for the swallowed graph-composition-failure defect

```
263 $it0 = new Itinerary('Vilnius', 'VNO', 2, 1, 500.0, 'Sunny');
264 $it1 = $it0->advance(graphEdge: new GraphEdge(
265     'e0', new GraphNode('VNO', 'Origin Hub', 'origin'), $nVNO, '
            Arrive', TripMetrics::empty()
266 ));
267 $advanceThrew = false;
268 try {
269     $it1->advance(graphEdge: new GraphEdge('bad', $nHTL, $nACT, '
            Mismatch', TripMetrics::empty()));
270 } catch (RuntimeException $e) {
271     $advanceThrew = true;
272 }
273 assertLaw(
274     '7. Free Category Inductor F(G) (Itinerary Fail-Fast)',
275     'Composition Failure Propagation',
276     'compose(p1, p2) = Left  =>  advance() throws, never silently
            ignores it',
```

```
277         $advanceThrew,
278         'Itinerary::advance()_throws_RuntimeException_when_the_induced_
                path_target_does_not_match_the_next_edge_source.'
279     );
```